

Formal proofs as guardrails for LLM coding agents

A workflow for developing large-scale systems when agents write the code

Research review — approaches, evidence from three prototypes, and a roadmap
July 2026

No formal-methods background assumed — the four concepts you need are on slide 3.

Agents now out-write our ability to check

- LLM agents produce code faster than humans can meaningfully review it. Human attention is the bottleneck — and it samples, it does not cover.
- Tests also sample: they check the runs you thought of. Concurrency bugs live precisely in the interleavings you did not think of.
- If we want agents to optimize systems autonomously (performance, cost), we need a way to say what must never break — and have a machine enforce it.

Economics: formal specs used to be written in addition to code, by the same scarce experts. With LLMs, translation into formal language is assisted, counterexamples are consumed by a tireless agent, and one spec is amortized over unbounded regenerations of the code.

The proposal

The developer writes down intent as invariants. Agents write and rewrite the code. Reasoning engines sit between them and arbitrate: every change must provably keep the invariants.

Code becomes cheap to regenerate. The spec becomes the durable asset.

All the formal methods you need today

Invariant

A sentence about the system that must be true at every moment. Example: “no request ever reads a database column that has been dropped.” You already write these — in comments, runbooks and post-mortems. Here they become machine-checkable.

Model

A small, faithful board-game version of your system: its states and legal moves (a request commits, a migration step runs). Small enough to explore, faithful enough that its bugs are your bugs.

Checker

A tireless adversary. It plays every possible ordering of moves against the model — millions of interleavings you would never think to test — hunting for one that breaks an invariant.

Counterexample

The checker's proof of failure: the exact move list that breaks the invariant. Not “something is wrong” but “this sequence of 9 steps ends with a stale read.” Perfect food for an LLM to repair against.

Three roles, one contract

DEVELOPER

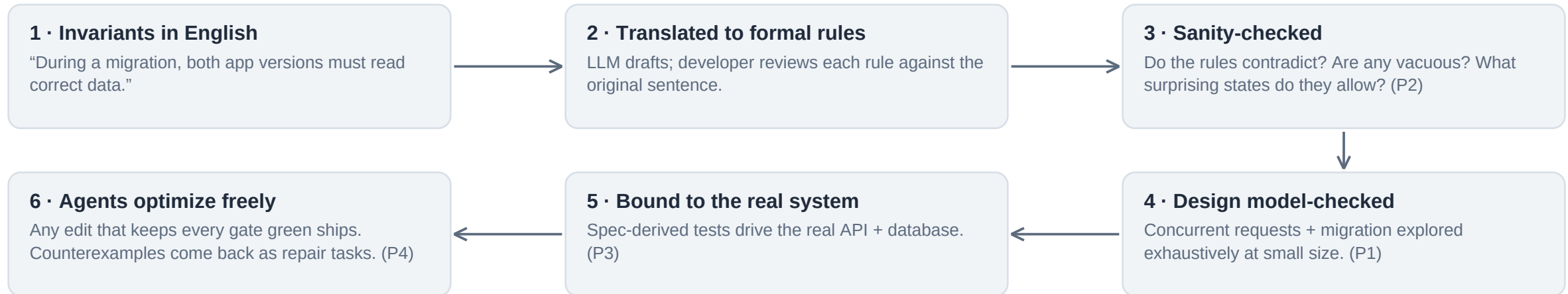
owns intent

LLM AGENTS

own implementation

REASONING ENGINES

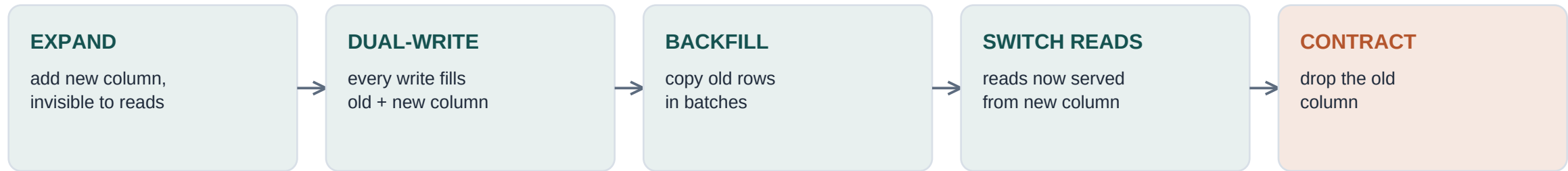
arbitrate



The contract: the spec is frozen for the agent — code edits can never weaken the gate; changing the spec is a human ceremony. Objectives (speed, cost) are optimized only inside the region the spec allows. The agent may make the system faster any way it likes; it may not make it wrong.

Changing the database schema while serving traffic

Why this problem: it is ubiquitous at scale, genuinely concurrent, and formally treated exactly once in the literature (Google F1, 2013). The popular open-source migration tools ship no correctness argument at all.



- Meanwhile: two versions of the application run side by side (rolling deploy). Version 1 has never heard of the new column.
- Meanwhile: user requests read and write the same rows the migration is copying — under snapshot isolation, so everyone sees a consistent-looking snapshot and conflicts surface only at commit.
- One protocol mistake in the choreography and some user reads return stale data — silently.

Three ways to put a formal fence around an agent

A · Design-first

Model the design (requests, migration steps) in a spec language; model-check it. Bind spec to code with generated tests.

- Covers whole-system concurrency — where the hard bugs live
- Cheapest; counterexamples are readable stories
- Weak spot: model and code can drift apart

B · Proof-carrying code

Write the implementation in a verification-aware language (Dafny, Rust+Verus); proofs live inside the code and re-verify on every edit.

- Strongest per-function guarantees; best-studied LLM+verifier combo (68–86% benchmark success)
- Weak spot: the database sits outside the verified boundary
- Locks the implementation language; slow verify cycles

C · Hybrid, layered ← chosen

A's design model + a typed, plain-English-reviewable invariant layer on top + selective code proofs only where optimization is riskiest.

- Each layer covers the gaps of the one below
- Team learns gradually: English rules first, models later, proofs last
- Every stage pays for itself before the next is needed

The checker beat us — twice

We wrote the migration protocol as a small model (Quint) with three one-line invariants, believing each version correct. Random simulation found counterexamples in under a second; symbolic checking confirmed them.

Bug #1 — the drain guard was in the wrong place

Rule as written: “backfill may not start until every app instance runs v2.”
Counterexample: dual-writes alone fill the new column, so the read switch fires with no backfill at all — while a v1 instance still lives. It writes the old column afterwards; a v2 user reads stale data.

Fix: the drain must gate the read switch too, not just the backfill.

Bug #2 — the textbook backfill query is wrong

Every guide says: backfill WHERE new_column IS NULL. Counterexample: during the rolling window a v2 instance dual-writes a row (new column now non-NULL), then a surviving v1 instance overwrites the old column only. The row is non-NULL but stale — and IS NULL never revisits it. The staleness survives to the switch.

Fix: backfill WHERE new IS DISTINCT FROM old; switch when all rows agree.

Both are real production failure modes of app-level dual-write migrations — and neither is in the literature or the OSS tools' docs. After the fixes: 30,000 random traces clean, and Apache verifies all executions up to 12 steps.

This is the workflow working as designed: the human states three one-line invariants; the machine finds the non-obvious consequences of the protocol.

Asking the logic engine: do my rules even make sense?

Before any model or code exists, the invariant set itself can be interrogated (prototype P2: a small CLI over the Z3 solver; rules are typed and plain enough to review sentence-by-sentence).

“Do any rules contradict?”

IMPOSSIBLE

Two developers add “at most one app version at a time” and “at least two during rolling deploys.” The solver returns the minimal conflicting pair — by name — out of the whole rule set.

“What states do the rules allow?”

SATISFIABLE

Enumerated example states reveal a forgotten guard: “dropping the column while an old binary still runs” was allowed. Nobody wrote a wrong rule — one rule was missing. The witness shows it.

“Does my claim follow from the rules?”

VALID / INVALID

“Contract never starts while v1 runs” → VALID: the rules entail it, no test suite needed. The same query later gates agent edits.

Each verdict is also emitted as JSON — the machine-readable half of the future agent loop.

The same invariant, enforced against real Postgres

Prototype P3: a real Rust API (two versions running side by side), a real Postgres database, the real five-step migration — exercised by generated concurrent traffic while the migration runs (Hypothesis test harness).

735

concurrent requests across all
migration phases

0

errors with the proven choreography

59

“column does not exist” errors per run
when the drain step is deliberately
skipped

1

genuine race found in the ‘modern’
instance itself (and fixed)

- The deliberately-broken run reproduces exactly the anomaly the model predicted — the formal invariant is load-bearing, not decorative.
- Bonus find: a check-then-act race — the app reads migration flags, then executes SQL a moment later; a contract commit in that gap 500s a fully-migrated instance. The class of bug that survives code review and unit tests.
- Design-level proof (P1) and real-system evidence (P3) now tell the same story about the same invariants.

Ten security rules everyone recognizes

The same workflow on a familiar domain: a content management system with user roles, draft articles, a review/publish lifecycle, and public access. Requirements start as stakeholder sentences in tickets:

inv_draft_visibility

"A draft is visible only to its author, editors, and admins."

inv_publish_staff_only

"Only editors and admins may publish."

inv_anonymous_published_only

"Anonymous visitors can read published articles and nothing else."

inv_deactivated_does_nothing

"A deactivated account can neither edit nor publish anything."

inv_archived_not_public

"Archived articles are not publicly accessible."

Lifecycle



Roles: anonymous, author, editor, admin — plus admin actions (demote, deactivate) that can fire at any moment.

Each sentence becomes one typed rule (LLM-drafted, reviewed next to the original). The rule NAMES travel through every later stage — oracle verdicts, model invariants, and the app's 403 responses all cite them.

The oracle at code-review time

Two tickets collide

IMPOSSIBLE

Security files SEC-482: “sessions expire within 15 minutes” (SOC2). Editorial UX files CMS-1201: “sessions last at least an hour.” Both get translated and merged. The solver flags the set as unsatisfiable and the unsat core names exactly those two rules — out of twelve — before any session code exists.

A rule silently kills a feature

INVALID

Legal adds LEG-77: “unpublished material is accessible only to the person who wrote it.” Nothing contradicts — the set stays CONSISTENT. But query the claim “an editor can view someone else's draft” → INVALID: under the combined rules, review-before-publish is dead. Every individual rule looks reasonable; the solver sees the consequence.

Both answers arrive at review time, from a query — not from a production incident or a bug report three sprints later.

This is the project's “query the invariants for contradictions and unexpected outcomes” step, running on real files in the repo: [examples/cms/invariants/](#).

The checker finds the stale-session race

Single-state rules can't see time. The model adds it: sessions cache the user's role at login; admin actions (demote, deactivate) run concurrently. One constant selects the design — and only one survives:

cms_live — re-check on every action

20,000 simulated traces clean; Apalache verifies all executions to depth 10.
Authorization at time of use is safe by construction.

cms_cached — trust the session snapshot

Counterexample in under a second: an author logs in, the admin deactivates the account, the still-open session submits content anyway. Other seeds: a demoted editor still publishes.

- The same check-then-act bug class the migration prototype caught in real Postgres — here it is authorization (stale JWT / cached claims), found by the checker before any code.
- The demo app (examples/cms/app, Rust) makes the knob real: AUTH_MODE=live passes the policy suite (5/5); AUTH_MODE=cached reproduces the model's counterexample over actual HTTP (2/2 stale-token violations — demoted and deactivated), and live mode refuses both with the named rules.
- Every 403 names the violated rule (inv_publish_staff_only, ...) — counterexamples stay machine-readable from solver to model to running server.

Features live in the model too

Safety invariants alone are gameable: a CMS that rejects every request satisfies all of them vacuously — an optimizing agent would happily “fix” bugs that way. So the model also specifies what must WORK, at three levels of strength (the first two run in examples/cms/model today):

1 · Scripted scenarios

run publishLifecycleTest: author drafts and submits, editor publishes, article ends published, no policy violation on the way. A model-level acceptance test — `quint test` fails the build if any step becomes impossible.

2 · Reachability witnesses

featSubmitted / featPublished: how many random traces exercise each feature — currently 28% and 1.3% of 3,000. Feature coverage for the model, and a tripwire: a guard change that kills a feature drops its witness count to zero.

3 · Temporal liveness

“Every submitted article is eventually published or returned.” Expressible in Quint/TLA+, needs fairness assumptions — the natural next level once safety and possibility are stable. Not yet wired in.

- Measured complementarity: the happy-path feature test passes in BOTH configurations — features don't catch the stale-session race, and the safety invariant doesn't catch a feature quietly dying. An agent gate needs both directions.
- The rule layer has a static analogue: a claim query staying SATISFIABLE/VALID is a feature check — the “editor can review drafts” INVALID verdict on slide 11 was exactly a feature regression, caught by a solver.

What guarantees the code matches the model?

By default: nothing. Model and code are separate artifacts, and drift between them is the best-documented failure mode in industrial formal methods. The guarantee is not found — it is BUILT, as a ladder where each rung narrows the gap and costs more than the one below:

5 • Proofs in / code from the spec

Verify the implementation itself (Rust+Verus, Dafny) or generate code + logging from the model (TraceLink/PGo style). The only true guarantees — bought selectively, for the riskiest code.

proof

4 • Trace validation

Log the running system in the model's vocabulary; replay the log against the model: "is this a legal behavior?" Strongest black-box link — must be designed in from day one, not retrofitted (MongoDB's 10-week lesson).

strong evidence

3 • Design-twin knobs & replayed counterexamples

The model's CHECK_AT_ACTION and the app's AUTH_MODE are the same decision; the model's counterexample became a real HTTP test and reproduced 2/2. Every model finding becomes a regression test.

evidence

2 • Spec-derived tests against the real system

Property/generated tests assert the same invariants over live traffic (migration harness: 735 requests, 0 errors; CMS policy suite 5/5). Samples behavior — high value, no proof.

evidence

1 • Shared vocabulary

One set of invariant names flows from tickets to rules to model to the app's 403 bodies. Proves nothing alone — but makes every failure at any rung traceable to a requirement.

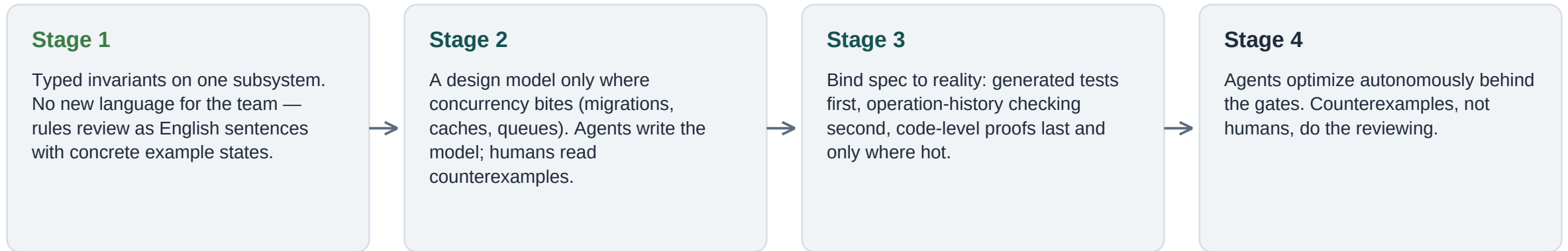
traceability

Assurance is a dial, not a boolean — and the agent loop re-runs rungs 1–4 on every edit, so drift is caught the moment it is introduced, not years later in an incident review.

What's new here

- No mainstream agent tooling gates edits on a formal checker today — “spec-driven development” products use prose specs. The gap between research verifier-loops and agent products is exactly where this sits.
- Two-layer specs: plain-English-reviewable typed rules on top (reliable LLM translation, human review), a temporal model underneath (real concurrency reasoning). Combines the only two approaches with strong evidence.
- First formal treatment (that we found) of app-level dual-write migration choreography — both P1 counterexamples are absent from the literature and from the OSS tools' documentation.
- Spec-as-fence, not spec-as-target: most verified-codegen work generates correct code once. Here the spec fences an autonomously evolving system — the agent optimizes an objective inside a frozen feasible region.

A learnable ramp — each stage pays for itself



Next on this project

- P4 — the closed loop: checker counterexample → LLM repair → recheck, reused at every layer (the one missing piece for milestone 4).
- History checking (Jepsen/Elle-style) on P3's recorded operations; multi-statement transactions in P1 to exercise snapshot-isolation write skew against migrations.
- Open questions: NL→spec fidelity at scale · trace-first specs (generate instrumentation, don't retrofit) · review UX for temporal counterexamples · module system for many small specs.

The one-sentence takeaway

Write intent once, formally, with machine help —
then let agents rewrite the code forever,
with a tireless adversary guaranteeing they never make it wrong.

Repo: `research/INDEX.md` (all notes) · `research/08-workflow-vision.md` (this argument, long form)

Prototypes: `p1-migration-model` (`./check.sh`) · `p2-invariant-oracle` (`./demo.sh`) · `p3-conformance-harness` (`./run_demo.sh`)

Worked example: `examples/cms` — the CMS from slides 10-12, every artifact runnable